

Computer Security Exam

Professors S. Zanero & M. Carminati & A. Barenghi

13/07/2023

Last (family) Name _____

First (given) Name _____

Matricola _____

Codice Persona	
-----------------------	--

Professor: ☐ Zanero ☐ Carminati ☐ Barenghi

Did you participate in the extra points practical test? ☐ Yes ☐ No

Do you want to withdraw? ☐ Yes ☐ No

Priority Grading ☐ Yes, Motivation:

[] No

Instructions

- **Duration:** 2.5 hours.
- The exam is composed of 17 pages. Check that you have all of them.
- Just as a cross-check, tell us whether you have completed challenges by appropriately putting an "X" mark.
- The exam is "closed book". The students will not be allowed to consult any **course material** and **notes**.
- **No extra devices** (e.g., phones, iPad) are **allowed**. You will be expelled if, at any time, you do not follow this rule.
- Shut down and store any electronic device. They will be subject to inspection if found and you may be expelled if you are found using one.
- You are not allowed to communicate with other students, and you will be expelled from the exam if you do.
- Please answer within the allowed space. Schemes are good; short answers are recommended.
- You can write in pen or pencil, any color, but avoid writing in red.
- No extra paper is allowed.
- **Always motivate your answer.**
- **If your final grade is below 12 (not considering the challenges), you will not be allowed to take the next exam call (even in the next session).**

- **DISTANCE MODE ONLY**

- On the computer, you will be allowed to open only the MS forms browser tab and the exam pdf. Any other application or document is prohibited.
- You will have to share your screen, leave your microphone open, and your webcam must frame both you and your workspace.
- You are responsible for having all the necessary technological equipment for the correct delivery of the exam.
- If you disable the screen sharing or the audio, you will be expelled from the virtual room and your exam will be voided.
- We will ask remote students to sustain a brief oral exam to confirm the evaluation of the written exams. This could be done for all students or for a subset, at the instructor's decision.
- Regarding the submission:
 - At the end of the exam, you will be given 10 minutes only to scan your documents and prepare to upload them.
 - After 10 minutes, you will be called by name and required to click on the submit button of the MS forms under the supervision of the supervisor of the virtual room. Any submission that will not follow this procedure (e.g., you have already submitted before being called or you are not ready to submit when called) will be voided.
 - We are not responsible for any technical issue in the submission (e.g., the exam pictures are not readable, the pdf is corrupted). We will evaluate only the uploaded readable documents.
 - We will evaluate only the uploaded **readable** documents.
 - USE YOUR PERSONAL CODE (CODICE PERSONA) ONLY AS THE NAME OF THE DOCUMENT (e.g., 12345678.pdf).

PLEASE, USE THE FORMATTING SPECIFIED IN THE QUESTION

READ CAREFULLY ALL THE POINTS OF EACH QUESTION BEFORE WRITING YOUR ANSWER

Exercise 1 - Introduction to Computer Security (4 points)

Consider a fingerprint door lock, an advanced version of a smart door lock that features biometric technology for enhanced security and usability. It combines the functionality of regular smart door locks with the added feature of a fingerprint scanner.

The key feature of a fingerprint doorbell is that it includes a fingerprint scanner. This scanner allows for keyless entry, giving access to only those individuals whose fingerprints have been previously scanned and stored in the system, eliminating the need for physical keys. The scanned fingerprints are typically stored locally on the device itself in a secured digital memory.

[2 points] 1. Describe two different relevant assets and related threats at risk in this scenario (Note that the threat must be related to the described asset). For each threat identified, also propose a security mitigation. Motivate all your answers. Solutions with no equal threats and assets will be evaluated more.

Asset	Threat	Security mitigation
Home/building/personal belongings/safety of people inside the building	Spoofing: Biometric systems can potentially be tricked using fake fingerprints made from materials like silicone. High-quality replicas can potentially fool less sophisticated sensors.	Multifactor authentication/increase the accuracy of the scanner
Scanned fingerprint	Data Breach: If the fingerprint data isn't stored securely, it could be potentially exposed in a data breach. This might not allow for replication of the fingerprint (as hashed data can't be reverse-engineered into a fingerprint), but it could be a privacy concern.	Scanned fingerprint must be stored hashed + device must be tamper proof.

[2 points] 2. One of the producers of fingerprint door locks installed in public areas decided to add a numpad to allow users with disabilities to authenticate. The producer designs the Numpad with an anti-peeping cover like the ones available at ATMs. Considering only the possible attacks (i.e., snooping, guessing, cracking) and mitigations seen at lecture to secure a passkey, discuss which would be the most likely attack and mitigation in the use case presented above.

Attack: Guessing
Mitigation: Not related to user
Complexity and change may help

Exercise 2 - Introduction to Cryptography (6 points)

[4 points] Consider the following user authentication mechanism, implemented on a server, which uses an asymmetric encryption algorithm, for which any 1024 bit string is a valid ciphertext :

1. Upon user enrollment, the user generates an asymmetric encryption keypair, and sends in cleartext the public key to the server administrator. The server administrator checks reliably that the public key corresponds to the user, and it hasn't been altered in transit, and stores the (user ID, public key pair) in the home directory of the user.
2. Upon a login attempt, the user picks a random 1024 bit string x and decrypts it with its private key, obtaining a string s . The user sends the triple (user ID, x, s) to the server
3. The server employs the public key corresponding to the user ID to encrypt s , and checks if it obtains the x it received from the user. If it matches, the user is authenticated.

Provide evidence of the soundness of this protocol, or show a practical attack and a corresponding countermeasure.

The authentication protocol is trivially broken: it is sufficient for an attacker to eavesdrop on the public key during phase 1, pick a random string s and encrypt with it obtaining the corresponding x . Swapping the encryption algorithm for an asymmetric signature algorithm and having the server choose the random challenge value x yields a valid authentication mechanism.

[2 points] Consider the usual Public Key Infrastructure (PKI) approach to certificate validation. Certificates are usually arranged in a tree, where each node is a certificate, and an edge from certificate A to certificate B states that the issuer of certificate B is the subject of certificate A.

What are the issues arising from changing the tree into a generic graph?

Can they be fixed by changing the validation process?

The two generalizations in moving from a tree to a generic graph are:

- Nodes with more than one predecessor: they correspond to certificates with more than one issuer. This requires a change in the validation process, as a decision should be taken if only some of the signatures on the certificate are valid.
- Non trivial cycles (cycles already exist on root CAs): a non trivial cycle requires the validation process to operate loop detection (e.g., by Floyd's turtle and hare algorithm), and implies that any certificate in a cycle should be regarded as self signed. Indeed, its validity depends, cyclically, on itself.

Exercise 3 - Binary Vulnerabilities (6 points)

```
00 #include <stdio.h>
01 #include <stdlib.h>
02 #include <string.h>
03 #include <unistd.h>
04 #include <fcntl.h>
05
06 typedef struct {
07     char username[40];
08     char password[20];
09     char secret[20];
10     int randomFd;
11 } user_t;
12
13 void welcome_user(user_t * user){
14     char message[52];
15     strcpy(message, "Ciao to ");           // len 8
16     strcat(message, user->username);       // len 8+40
17     strcat(message, "\n");                // len 8+40+1 == 49 < 52, ok
18     printf(message);
19     return;
20 }
21
22 int main() {
23     user_t user;
24
25     printf("Insert your name: ");
26     fflush(stdout);
27     read(0, &user.username, 40);          // 0 == stdin
28     printf("Insert your password: ");
29     fflush(stdout);
30     read(0, &user.password, 20);          // 0 == stdin
31
32     user.randomFd = open("/dev/urandom", O_RDONLY);
33     read(user.randomFd, &user.secret, 20);
34
35     welcome_user(&user);
36     if (!strcmp(user.password, user.secret, 20)){
37         printf("You're a lucky user!\n");
38         system("/bin/sh");
39     }else{
40         printf("The Mirage Island isn't visible today :(\n");
41     }
42
43     return 0;
44 }
```

Assume that:

- Environment variables are located in the highest addresses.
- The program is compiled for the **x86 architecture** (32 bit) and for an environment that adopts the usual **cdecl** calling convention. Furthermore, assume that **no compiler-level or OS-level mitigations** against the exploitation of memory corruption errors are present (unless specified otherwise).
- **fflush(stdin)** is used to correctly manage the scanf(): **It is not relevant for the exploitation process, you can ignore it for the questions below.**

- `char * strcat(char *restrict s1, const char *restrict s2)` appends a copy of the null-terminated string `s2` to the end of the null-terminated string `s1`, then adds a terminating `'\0'`. The string `s1` must have sufficient space to hold the result.
- `ssize_t read(int fd, void *buf, size_t nbyte)` attempts to read `nbyte` bytes of data from the object referenced by the descriptor `fd` into the buffer pointed to by `buf`. No extra bytes (*nor the string terminator*) are added by the function.

[1 point] 1. The program is affected by typical buffer overflow and format string vulnerabilities. Complete the following table, focusing on a vulnerability per row.

Vulnerability	Line	Motivate and Modify the line to solve the detected vulnerability
Buffer Overflow	[0.25] Line 16; With the aid of 27, which does not put a terminator '\0'.	See Slides [0.25]
Format String	[0.25] Line 18;	See Slides [0.25]

[2 points] 2. **Focus only on the stack-based buffer overflow(s) you found.** Write an exploit for this vulnerability that **must execute the following shellcode**, composed of 8 bytes, which opens a shell: `0x20 0x30 0x40 0x50 0x60 0x70 0x80 0x90`.

2.a) Describe **all the steps and assumptions** required to successfully exploit the vulnerability. Include also any assumption on how you must call and run the program: e.g., the values for the command-line arguments required to trigger the exploit correctly and/or environment variables (if any), the input provided during the execution, if multiple executions are necessary.

You can exploit the buffer overflow on the variable message of the function welcome_user. The overflow happens with the strcat at line 16, combined with the fact that the variable user.username can be filled with 40 chars!= '\0' by the read of line 27. This way, the strcat will copy both user.username and user.password inside message, causing the overflow. We can fill user.username with a NOPsled and the shellcode (with a total of 40 non-zero bytes) and put the desired return address inside user.password.

*User.username = "\x90\x90\x90" ... + shellcode
User.password = "A"*8 + target_address*

ASLR and NX must be disabled unless you take further assumptions. The return address should not contain zeroes, otherwise they would stop the strcat before the full address is copied!

2.b) Make sure that you show how the exploit will appear in the process memory with respect to the stack layout right **before and after the execution of the vulnerable line** during the program exploitation showing:

- Direction of growth and high-low addresses;
- The name of each allocated variable;
- The content of relevant registers (i.e., EBP, ESP);
- The functions stack frames.

Show also the content of the caller frame.

	Before the vulnerable line			After the vulnerable line	
Low addresses					
^			Function Frames		
	message[0:3] "Ciao"	<- ESP	welcome_user()	message[0:3] "Ciao"	
	message[4:7] " to "		welcome_user()	message[4:7] " to "	
	message[8:11]		welcome_user()	0x90 0x90 0x90 0x90	<- target_address
	...		welcome_user()	...	
	message[44-47]		welcome_user()	0x60 0x70 0x80 0x90	
	message[48-51]		welcome_user()	not_relevant	
	sEBP	<- EBP	welcome_user()	not_relevant	
	sEIP		main()	~target_address	
	&user		main()	not_relevant	
	user.username[0-3]		main()	0x90 0x90 0x90 0x90	
	...		main()	...	
	user.username[36-39]		main()	0x60 0x70 0x80 0x90	
	user.password[0-3]		main()	not_relevant	
	user.password[4-7]		main()	not_relevant	
	user.password[8-11]		main()	~target_address	
	...		main()	...	
	user.password[16-19]		main()	not_relevant	
	user.secret[0-3]		main()	not_relevant	
	...		main()	...	
	user.secret[16-19]		main()	not_relevant	
	user.randomFd		main()	not_relevant	
	sEBP		main()	not_relevant	
	sEIP		main()	not_relevant	
	Argc		main()	not_relevant	
	Argv		main()	not_relevant	
High addresses					

[2 points] 3. **Focus only on the format-string vulnerability you found.** Consider ONLY the correctly implemented “NX” mitigation technique. Someone told us that you can use this vulnerability to obtain a shell, even with the NX mitigation enabled! Rumours say there might be **two working exploits** that do not require neither ret2libc nor ROP. If the rumours are true, explain the exploits, otherwise explain why the vulnerability cannot be exploited. Describe **all the steps and assumptions** required to successfully execute the exploits.

1) We could use the format string to overwrite the return address of either welcome_user or main with the address (in .text) of the code that prepares the call to system(“/bin/sh”). Being the address in .text, we will not have issues with NX. [1 pt].

2) We could use the format string to overwrite the content of user.secret with a short string like “A\0” and just input “A\0” in user.password. This way we will pass the check! [1 pt].

In both cases, we need to write the format string so that it does not contain the byte ‘\0’, or the strcat will ruin our exploit. The greatest danger are zeroes in the target addresses (not in “what to write” as it will be ascii!). Similarly, in both cases, we need to assume ASLR disabled or we will not know the stack addresses (addr.s of ret address and user.secret, respectively).

[1 points] 4. Consider ONLY the correctly implemented “ASLR” mitigation technique. Discuss whether the three exploits identified above can still work with this active mitigation. Both in the positive and in the negative case, **explain** why writing explicitly what you would be missing.

Focusing on the first attack and assuming to be a “local attacker” rather than a remote one, could you slightly modify the exploit technique to bypass ASLR?

None of the three exploits above would work with ASLR active. 1) You would not know the shellcode address to jump to. 2) You would not know the address of the return address 3) You would not know the address of user.secret.

The way to get around ASLR would be to insert the shellcode in a huge Environment Variable with a very long NOPsled. In that case, by jumping at an approximate stack address, you will highly increase the probability of landing on your shellcode.

Exercise 4 - Web Vulnerabilities (6 points)

StoryShelf.com is a website where users can freely read their favourite books online. It offers an extensive collection of books from various genres, spanning from 1950 onwards.

The relevant code of the application is the following:

```
00 app.route("/book/<book_name>") # /book/<book_name>
01 def search_item(request, book_name=None):
02     if request.method != "GET":
03         return abort(403)
04     if book_name is None:
05         return abort(403)
06     if page not in request.args.keys():
07         page = "0"
08     else:
09         page = request.args["page"]
10     check_query = "SELECT * FROM BOOKS WHERE title='" + book_name + "%';"
11     result = db_execute_query(check_query)
12     if len(result) != 1:
13         return abort(403)
14     page_path = "books/" + book_name + "/" + page
15     if exists(page_path):
16         f = open(page_path, "r")
17         page_content = f.read()
18         f.close()
19     else:
20         page_content = "Page not found!"
21     return render_template("book.html", book=book_name, page=page_content, page_num=page)

29 # book.html
30 <html>
31     ...
40     <h1>{{book}}</h1>
41     <h2>Page: {{page_num}}</h2>
42     <p>{{page}}</p>
43     ...
60 </html>
```

The relational database schema used by the website is the following:

Table: **BOOKS**

book_id (Primary Key) (Integer) (Auto increment)	title (String)	author (String)	year (Date)
--	-------------------	--------------------	----------------

Assume the following:

- The function **render_template** **does not perform any sanitization of the input.**
- The **DBMS** does not support stacked queries (e.g., "SELECT ... ; INSERT INTO ... ;" fails if executed)

More details:

- The function **render_template** generates an html page with optional parameters that are placed in the html code.
- The function **open** opens a file in the filesystem given the path of the file (1st parameter) and the mode of operation (2nd parameter). The mode of operation can be "r" to read the file, or "w" to write in the file.

1. [3 points] Fill out the following table by answering the questions for each class of vulnerabilities. While answering the questions, be **exhaustive, concise, and straight to the point**. If you believe there might be a vulnerability, specify all the corresponding line/s of code.

<i>Vulnerability class</i>	<i>Is there a vulnerability of this class in the code above? How could an adversary exploit it?</i>	<i>If the vulnerability is present, explain the simplest procedure to remove it.</i>
SQL Injection (SQL-I)	<i>Line 10. There's a SQL-Injection vulnerability. The attacker may leak data from the DB. Moreover, the attacker can only perform a blind SQL-Injection.</i>	<i>Use prepared statements.</i>
cross-site scripting (XSS) Specify if reflected, stored, DOM...	<i>Page number is not sanitised and can be exploited to perform a reflected XSS. Line 41.</i>	<i>The simplest procedure would be to apply escaping/filtering to the message.</i>
Cross-site request forgery (CSRF)	<i>There is no state changing action. Thus, there's no need for a CSRF token.</i>	<i>See slides (CSRF tokens, etc.).</i>

[1.5 points] 2. One of your friends sent you a link to StoryShelf.com. Once clicked, an alert pops up saying: "Why would you read a book if you can watch the movie?!?". Can you reproduce the exploit? Explain all the steps you'd take to reproduce the exploit, including the final code you'd write.

```
page = <script>alert("Why would you read a book if you can watch the movie?!?");</script>
```

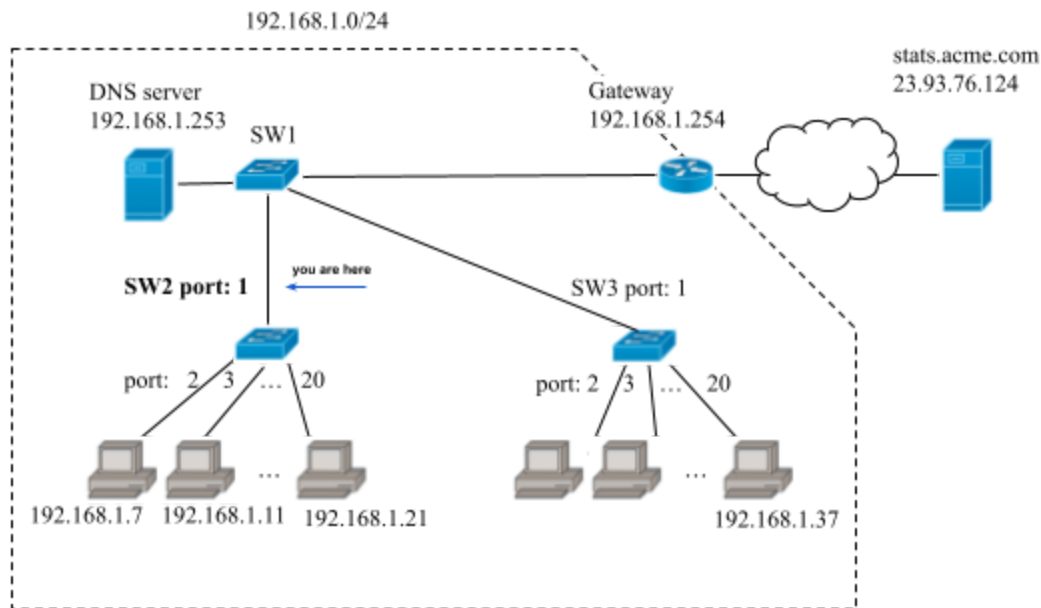
[1.5 points] 3. Can you spot any additional vulnerability w.r.t. the ones mentioned in the table of point 1? If yes, please describe the vulnerability you found and provide an exploit that exploits it.

```
There's a path traversal vulnerability at line 15. The user can read an arbitrary file in the filesystem.
```

```
page = "../../../etc/passwd"
```

Exercise 5 - Network Security (6 points)

CodeCraft is a company that develops various kinds of applications for third parties (mobile applications, web applications, etc.). The company has recently hired several new developers, and has moved its headquarters in a new building. Their internal network had to be redesigned from scratch, and this is what it looks like:



The office building has two floors: there is a main switch that connects the two floors, and each floor has a dedicated switch to which developers' machines are connected. A few servers, providing internal services, are connected to the main switch: for simplicity, we only put the DNS server in the network diagram, which is used to provide internal address resolution and custom domain names inside the network.

One of the developers observed some weird behavior on the browser of his machine. He decided to alert the internal security team, which accessed a trace of packets flowing through Port 1 of the switch SW2, which connects to switch SW1. This is the trace of packets that they found:

Source	Destination	Protocol	Packet Content
192.168.1.7	192.168.1.253	DNS	Query ID 0x1000 A stats.acme.com
192.168.1.21	157.240.203.174	HTTP	GET /
23.93.121.244	192.168.1.253	DNS	Response ID 0x01 stats.acme.com A 66.117.117.117
157.240.203.174	192.168.1.21	HTTP	200 OK
23.93.121.244	192.168.1.253	DNS	Response ID 0x02 stats.acme.com A 66.117.117.117
192.168.1.21	157.240.203.63	HTTP	GET /rsrc.php/v3/[...]3xuv5mvUF12.css?_nc_x=Ij3Sa8lg3Kz
23.93.121.244	192.168.1.253	DNS	Response ID 0x03 stats.acme.com A 66.117.117.117
23.93.121.244	192.168.1.253	DNS	Response ID 0x04 stats.acme.com A 66.117.117.117
23.93.121.244	192.168.1.253	DNS	Response ID 0x05 stats.acme.com A 66.117.117.117
23.93.121.244	192.168.1.253	DNS	Response ID 0x06 stats.acme.com A 66.117.117.117
192.168.1.21	157.240.203.63	HTTP	GET /rsrc.php/v3/y-[...]SArU.js?_nc_x=Ij3Sa8lg3Kz
23.93.121.244	192.168.1.253	DNS	Response ID 0x07 stats.acme.com A 66.117.117.117
23.93.121.244	192.168.1.253	DNS	Response ID 0x08 stats.acme.com A 66.117.117.117
23.93.121.244	192.168.1.253	DNS	Response ID 0x09 stats.acme.com A 66.117.117.117
23.93.121.244	192.168.1.253	DNS	Response ID 0xA stats.acme.com A 66.117.117.117
[...]			
23.93.121.244	192.168.1.253	DNS	Response ID 0x2315 stats.acme.com A 66.117.117.117
192.168.1.253	192.168.1.7	DNS	Response ID 0x1000 stats.acme.com A 66.117.117.117
23.93.121.244	192.168.1.253	DNS	Response ID 0x2316 stats.acme.com A 66.117.117.117
23.93.121.244	192.168.1.253	DNS	Response ID 0x2317 stats.acme.com A 66.117.117.117
23.93.121.244	192.168.1.253	DNS	Response ID 0x2318 stats.acme.com A 66.117.117.117
192.168.1.7	66.117.117.117	HTTP	GET / HTTP/1.1
66.117.117.117	192.168.1.7	HTTP	200 OK

1. [2 points] Look at the sequence of packets. Identify the ongoing attack and explain how it works.

The attack is a DNS cache poisoning. [See slides for explanation]

2. [1 point] Who is the victim of the attack?

The victim is any machine that is using DNS as a recursive (local) DNS server. One of the (successful) victims of the attack may be 192.168.1.7 (we can see from the traffic that it connects to the “wrong” website...).

3. [1 point] Can you provide all the information that you have about the attacker (e.g., IP and network location)?

IP: no, it's spoofed (note: it CAN be spoofed is a wrong answer, it IS for sure spoofed, that's how the attack works)

Location: Someone connected to SW1.

4. [2 points] Assume to be the administrator of the network 192.168.1.0/24. Select the single solution that better mitigates the ongoing attack in this scenario. For each option, reply with Yes or No and briefly justify them.

1. Replace SW1, SW2, and SW3 with firewalls. Add to the firewall a rule that denies all requests from and to IPs 157.240.203.174 and 157.240.203.63. Move the local DNS server to the DMZ.
2. Replace SW1 with a firewall. Allow only known hosts to communicate from each subnetwork.
3. Move DNS queries to TCP, removing the amplification factor.
4. Upgrade server *stats.acme.com* (23.93.76.124) to HTTPS.

- 1) NO, with this solution you are only blocking access to 157.240.203.63 and 157.240.203.63 (Instagram) to users in the network.*
- 2) YES, as the attacker cannot spoof anymore the authoritative DNS reply to the local DNS server.*
- 3) NO, there is no amplification factor and the attack would still be feasible.*
- 4) NO, the administrator of the network does not have control of stats.acme.com.*

Exercise 6 - Malware Security (6 points)

101 Inc. was recently targeted by a new malware named **Crudelia**. We managed to extract a portion of the code from this malicious software and identified some intriguing functions. The most notable sections are shown in the following C pseudo-code.

```
00 void evil() {
01     // REDACTED
02 }

03 int main () {
04     unsigned long time_1 = 1691755932L;
05     while (time(NULL) < time_1) {
06         sleep(2);
07         continue;
08     }
09     evil();
10 }
```

The output of the command `ltrace` on the crudelia malware is the following one (e.g. `ltrace ./crudelia`).

```
time(0) = 1688823812
sleep(2) = 0
time(0) = 1688823814
sleep(2) = 0
time(0) = 1688823816
sleep(2) = 0
time(0) = 1688823818
...
```

1) [2 points] Which stealth technique does this malware employ? Please explain how this is implemented in this particular case. Please note: naming wrong techniques will make you lose points.

The malware implements a time-bomb/dormant code stealth technique. It is waiting for a certain timestamp before doing evil actions.

2) [2 points] What type of analysis would you apply to the Crudelia malware? Please provide a detailed motivation of your answer and specify all the assumptions you have made.

It is possible to apply static analysis. In fact, the malware is not obfuscated. It is possible to apply dynamic analysis only after the date when the time bomb activates.

Additional answer: it is possible to modify the binary to force the execution of the malicious operations. This can be done by patching the assembly instructions of the sample, or manually modifying in gdb the result of the time function call (this answer is provided for the sake of completeness and is not strictly required, but it would be highly appreciated).

Let consider a variant of the Crudelia malware shown in the following C pseudo-code.

```
00 void evil() {
01     // REDACTED
02 }
```

// From man ptrace

The `ptrace()` system call provides a means by which one process (the "tracer") may observe and control the execution of another process (the "tracee"), and examine and change the tracee's memory and registers. `PTRACE_TRACEME` is used by a process to ask its parent to debug it. Remember that Linux allows a process to be traced (or, debugged) by a single process at the same moment. On error, `ptrace` returns -1.

```
03 int main () {
04     int ptrace_result = ptrace(PTRACE_TRACEME, 0, 0, 0);
05     if (ptrace_result == -1) {
06         puts("Daga kotowaru!");
07         exit(-1);
08     }
09     unsigned long time_1 = 1691755932L;
10     while (time(NULL) < time_1) {
11         sleep(2);
12         continue;
13     }
14     evil();
15 }
```

3) [2 points] Compared to the original version, what new stealth technique does this malware implement? Is it possible to analyze this variant using dynamic analysis? Why? Please specify all the assumptions you have made. Please note: naming wrong techniques will make you lose points.

The variant implements anti-debugging techniques. In fact, the function call `ptrace(PTRACE_TRACEME, 0, 0, 0)` returns -1 when the program is already being debugged. So, it is not possible to do dynamic analysis using the basic techniques analyzed in class.

Additional answer: it is possible to modify the binary to force the execution of the malicious operations. This can be done by patching the assembly instructions of the sample, or manually modifying in gdb the result of the `ptrace` function call (this answer is provided for the sake of completeness and is not strictly required, but it would be highly appreciated).